Pau Casesnoves
Centre Integrat de Formació Professional

PROVA D'AVALUACIÓ UNITAT 1

Curs	26/27	**Grup**	IFC33C	**Data**		**Qualificació**	
Mòdul	Desenvolupament web en entorn servidor						
Alumne/a							

El servei de reprografia de la biblioteca

La biblioteca desenvolupa amb Java i Spring Boot un servei d'impressió. Cada petició HTTP pot incloure diversos documents. El sistema prepara cada document, en gestiona la impressió i registra la traça de l'operació.

Respon amb codi Java, anotacions i justificacions. Pots ometre imports, paquets i codi repetitiu, i simular operacions amb missatges. No cal implementar controladors. Els apartats es valoren separadament: suposa correctes les altres peces. Pots continuar al revers indicant l'apartat.

1. Preparació de documents — 1,5 punts

La interfície `PreparadorDocument`, amb el mètode `String preparar(String nomDocument)`, té dues implementacions gestionades per Spring: `PreparadorText` i `PreparadorImatge`.

Una dependència de tipus `PreparadorDocument` sense preferència concreta ha de rebre el de text. `ServeiCartelleria`, compartit per tota l'aplicació, necessita el d'imatge.

a) Escriu la interfície i les declaracions de les implementacions, amb anotacions i signatures; omet el cos de `preparar`. **(0,75 punts)**

b) Escriu la declaració, el camp i el constructor de `ServeiCartelleria`. Explica quin preparador rebria un servei sense preferència concreta. **(0,75 punts)**

2. Marca d'aigua opcional — 1,5 punts

`ServeiPreparacio` necessita sempre un `PreparadorDocument`. Algunes instal·lacions també disposen d'un component `MarcaAigua`, amb el mètode `String aplicar(String contingut)`, però en altres instal·lacions aquest component no està registrat a Spring.

L'aplicació ha d'arrencar en tots dos casos. El mètode `String preparar(String nomDocument)` de `ServeiPreparacio` obté primer el contingut del preparador. Si hi ha marca d'aigua, l'aplica; si no, retorna el contingut sense modificar.

Criteris de qualificació: Proposta: punts indicats a cada apartat. Total: 10.

a) Escriu la declaració del servei, els camps i el constructor per a la dependència imprescindible. **(0,5 punts)**

b) Mostra com rep la col·laboració de `MarcaAigua` sense exigir que el component existeixi. **(0,5 punts)**

c) Implementa `preparar` i explica què ocorre amb la injecció i amb el resultat quan no hi ha cap component `MarcaAigua`. **(0,5 punts)**

3. Traça d'una operació — 1,5 punts

Cada petició HTTP que gestiona impressions necessita un `ContextImpressio` amb un identificador aleatori, accessible amb `String getId()`. Tots els documents i missatges de la mateixa petició han de compartir aquest identificador. Una altra petició ha de tenir un identificador diferent, encara que provingui del mateix navegador.

`ServeiImpressio` i `RegistreImpressio` són serveis compartits per tota l'aplicació i es creen a l'arrencada, quan encara no hi ha cap petició HTTP. Tots dos reben directament una dependència de tipus `ContextImpressio` al constructor i la conserven en un camp. Durant l'atenció de la petició consulten `getId()` sobre aquesta dependència; no es passen l'identificador com a paràmetre.

a) Escriu `ContextImpressio`, amb la creació de l'identificador, el getter i la configuració necessària perquè satisfaci aquests requisits. Pots emprar `UUID.randomUUID().toString()`. **(0,5 punts)**

b) Escriu els camps i constructors pertinents dels dos serveis, amb les seves anotacions. No cal escriure la resta dels mètodes. **(0,5 punts)**

c) Justifica per què els serveis es poden crear abans de la primera petició i, després, consultar el context corresponent. Indica quins identificadors veuen dins una mateixa petició i en dues peticions diferents. **(0,5 punts)**

4. Un treball per document — 1,5 punts

Per a cada document, `ServeiImpressio` ha d'obtenir un `TreballImpressio` nou i gestionat per Spring. Dos documents de la mateixa petició necessiten dos treballs diferents, encara que comparteixin el context de l'apartat anterior. El servei continua sent una única instància compartida.

La classe ja té aquest codi; copia només la declaració i les anotacions que hi afegiries:

```
public class TreballImpressio {
    private final String id = UUID.randomUUID().toString();

    public String executar(String nomDocument) {
        return id + ": " + nomDocument;
    }
}
```

Criteris de qualificació: Proposta: punts indicats a cada apartat. Total: 10.

La petició dels treballs al contenidor s'ha de concentrar en una classe `FabricaTreballs`, amb el mètode `TreballImpressio nouTreball()`. El servei rep aquesta fàbrica i la utilitza cada vegada que processa un document. Ni el servei ni la fàbrica han de construir manualment els treballs amb `new`.

a) Configura `TreballImpressio` perquè cada petició d'aquest objecte al contenidor obtingui una instància nova. **(0,5 punts)**

b) Escriu `FabricaTreballs` i el fragment de `ServeiImpressio` que rep la fàbrica i implementa `String imprimirDocument(String nomDocument)`, obtenint un treball i retornant-ne el resultat d'`executar`. **(0,75 punts)**

c) Explica per què injectar directament un únic `TreballImpressio` al constructor del servei i conservar-lo no compliria el requisit. **(0,25 punts)**

5. Connexió amb la impressora — 1,5 punts

`MotorImpressora` és un component compartit que estableix una connexió costosa durant la seva construcció. Ofereix el mètode `String enviar(String contingut)`. La connexió només s'ha de preparar quan s'envia la primera impressió; les impressions següents reutilitzen el mateix component.

`ServeiEnviament` també és compartit i s'ha de crear a l'arrencada. Rep `MotorImpressora` pel constructor. La creació del servei no ha de provocar la connexió amb la impressora; el seu mètode `String enviar(String contingut)` delega en el motor.

a) Escriu la declaració de `MotorImpressora` amb la configuració necessària. Simula la connexió amb un missatge al constructor i implementa `enviar` retornant un text. **(0,5 punts)**

b) Escriu `ServeiEnviament`, incloent el camp, el constructor i el mètode de delegació. **(0,5 punts)**

c) Explica quan es construeix el motor i per què la configuració de la injecció al servei és rellevant per complir el requisit. **(0,5 punts)**

6. Preparació i alliberament de recursos — 1,5 punts

El component compartit `RegistreRecursos` gestiona un recurs propi on anota incidències de les impressions. Una vegada construït i amb les dependències ja proporcionades, ha de preparar aquest recurs abans de començar a prestar servei. Quan el context de Spring es tanqui ordenadament, l'ha de tancar.

Pots simular la preparació i el tancament amb missatges; no cal programar l'accés a fitxers.

Criteris de qualificació: Proposta: punts indicats a cada apartat. Total: 10.

a) Escriu la declaració del component i el mètode que prepara el recurs en el moment indicat. **(0,5 punts)**

b) Escriu el mètode que allibera el recurs en el tancament ordenat. **(0,5 punts)**

c) Explica en quin moment s'executa cadascun dels dos mètodes. S'executarien abans i després de cada impressió? Justifica la resposta. **(0,5 punts)**

7. Data i hora dels registres — 1 punt

El component `SegellTemps` ha de rebre pel constructor un `java.time.format.DateTimeFormatter` amb el patró `dd/MM/yyyy HH:mm:ss`. La classe del formatador pertany a la biblioteca Java i no la pots modificar.

La manera d'obtenir el formatador ha de quedar centralitzada en una classe `ConfiguracioReprografia`. Spring ha de gestionar l'objecte resultant i proporcionar-lo als components que el necessiten; `SegellTemps` no ha de crear el seu formatador.

Per crear l'objecte, l'API Java ofereix `DateTimeFormatter.ofPattern(String patro)`. Per obtenir la data i l'hora actuals amb aquest format, pots usar `LocalDateTime.now().format(formatador)`.

a) Escriu `ConfiguracioReprografia` amb el codi necessari perquè el formatador configurat quedi disponible al contenidor. **(0,5 punts)**

b) Escriu `SegellTemps`, amb el camp, la injecció i el mètode `String ara()`. Explica on canviaries el patró si la biblioteca volgués una altra presentació de la data. **(0,5 punts)**

Criteris de qualificació: Proposta: punts indicats a cada apartat. Total: 10.